ESSUN: A SPECIFICATION-DRIVEN, MULTI-FIDELITY SIMULATOR FOR SCALE-UP NETWORKS

Li Chen
https://github.com/li-ch/essun

ABSTRACT

Scale-up networks are the GPU-to-GPU fabrics behind LLM inference: Ultra Ethernet, UnifiedBus, UALink, and NVLink-class systems. Teams choose among these fabrics with one-off simulators and vendor numbers: which protocol to adopt, which topology to wire, which collective algorithm to schedule. The comparisons those tools produce are not apples-to-apples, and we could not reproduce published comparisons from the systems we examined, nor did we find a published cross-fabric comparison that states its inputs completely. We built essun, an MIT-licensed simulator, around one idea: one executable, provenance-carrying specification drives every fidelity, from closed-form analytical models to packet-level simulation, so results stay comparable and every number traces to an immutable run record. This white paper describes our approach, what we implement and test today (712 tests; a UEC 1.1 profile with 445 anchored requirements and 67 free parameters; three simulator engines behind one manifest, with the uet-htsim path ratified as the third backend; a cross-standard comparison; a complete 30-cell analytical design-space atlas), and what we have not done yet (hardware calibration, UALink and CXL ingestion). Both reference runs reproduce byte-for-byte from a fresh clone with one command.

1 The Problem

LLM inference runs on GPU pods whose fabric decides what the pod can serve. Inside one pod, the scale-up network carries every tensor-parallel collective between GPUs, and the current contenders are Ultra Ethernet [3], UnifiedBus, UALink, and NVLink-class systems [2]. The protocol a team adopts, the topology it wires, and the collective algorithms it schedules together shape serving cost and serving latency.

Fabric decisions today rest on one-off simulators and vendor numbers, and that base fails in four specific ways. The first failure is comparability: each comparison is its own simulator. Every team writes its own model of the fabric, makes its own modeling assumptions, and reports its own numbers. Nothing holds the two models equal, so results are not apples-to-apples even when the workloads look the same.

The second failure is reproducibility: we could not reproduce published comparisons from the systems we examined, and we found no published cross-fabric comparison that states its inputs completely. One-off simulators drift with the code around them. Seeds, topology versions, and configuration files go unpinned, so a rerun months later gives different numbers, and the record offers no way to tell which run is right.

The third failure comes from the standards themselves. Specifications such as UEC 1.1 mix normative requirements with choices each implementer makes [3]. Any one-off simulator must fill every gap to produce a number, and it fills the gaps with invented values that are invisible in the output. The result carries an authority the inputs never had.

The fourth failure appears when two simulators disagree. The common move is to tune parameters until the curves look similar, but that agreement is cosmetic. The modeling difference that produced the gap stays hidden, and the tuned parameters now encode an error.

We ask one question in response: can a single executable specification drive analytical, collective-level, and packet-level simulation of the same workload on the same fabric, with every published number traceable to an immutable run record?

We built essun to answer it. In essun, one experiment manifest runs the same workload across fabrics and fidelities. Every standard-derived value carries a source anchor or stands as an explicit free parameter. Run records are immutable, and both reference runs replay byte-for-byte from a fresh clone. When backends disagree, we decompose and classify the gap; we never tune parameters to force agreement.

The repository backs each claim with artifacts. The test suite holds 712 passing tests, and the specification profiles add provenance on top: the UEC 1.1 profile carries 80 page-hashed source anchors, 445 anchored requirement objects, and 67 declared free parameters, and it passed an owner expert review. The UnifiedBus profile derives from 5 public papers with 53 anchors, and the cross-standard comparison table shows a delta of 0 under shared parameters. Beyond the profiles, we now run three independent simulator engines behind one manifest: the L0 analytical family, the pinned ASTRA-sim analytical frontend, and the uet-htsim congestion-management path, which we ratified as the third backend (ADR-0004); we also built the csg-htsim packet-level frontend, and it reproduces in Docker. Our analytical design-space atlas covers all 30 cells (five rank classes by six algorithm families) at E0 level, and our 26 PROPOSED calibration candidates compare model identities against published measurements. The clearest artifact of all is the recorded cross-fidelity comparison: we explain a 7.404x completion-time disagreement between two simulators down to zero residual, without touching any parameter.

This white paper makes four contributions. We argue that fabric comparisons need one executable, provenance-carrying specification in place of one-off simulators. We present the design of essun: the SpecIR specification format, one manifest across fidelities, exact-arithmetic identities, capability declarations, immutable run records, and the classify-don't-tune discipline. We inventory what we implement and test today, with an evidence label on every row. We close with the decomposition of one 7.404x cross-fidelity gap to zero residual, which we obtained without touching any parameter.

2 The Approach

essun rests on five ideas. Each idea exists in code with tests behind it, and section 3 states exactly how far we have verified each one.

SpecIR: one specification, full provenance. We represent a fabric profile as an executable specification, the *SpecIR*. Every value that comes from a standard document keeps a source anchor: the section, the page, and a SHA-256 hash of that page. Every value the standard leaves open becomes a declared free parameter with a stable identifier. There is no middle category: every number in essun either carries an anchor to a cited source or is an explicit free parameter, so a reader can tell which is which in any output. When a standard is license-restricted, we do not ingest it at all; license review precedes ingestion, every time.

Every fidelity, one manifest. Each experiment manifest names the workload, the fabric profile, the fidelity, and the observables to record. The same manifest drives three separately-authored simulation engines: the L0 analytical family (six models: ring, crossbar, 2D torus, recursive doubling, phase collectives, and L0.5), the pinned ASTRA-sim analytical frontend [1], and the uet-htsim congestion-management path, which we ratified as the third independent backend (ADR-0004) after it passed all four adapter stages. The csg-htsim packet-level frontend inside the pinned ASTRA-sim is a fourth path in progress: we have built it and smoke-tested it in Docker, and it reproduces through a recorded seed patch; wiring its adapter remains scheduled work. We pin every external backend to an exact commit and build it through Docker with a recorded digest: ASTRA-sim at commit `4ce9ecf` (MIT), csg-htsim at commit `67cbbbb1` (BSD-2-Clause), and uet-htsim at commit `65ea315b` (BSD-2-Clause) [4]. Because every fidelity reads the same manifest, a comparison across fidelities holds everything but the fidelity constant.

Identities that hold exactly. Observables come in two kinds: conservation quantities, such as wire bytes moved by a collective, and performance quantities, such as collective completion time (O-CCT). The L0 family computes in exact integer arithmetic, so conservation identities hold bit-exactly. This exactness matters most when two backends disagree: an identity that both sides must satisfy separates real modeling differences from arithmetic noise.

Capability declarations that fail loudly. Each backend declares which observables it can produce. Asking a backend for an observable outside its declaration aborts the run and records the observable as unsupported. The alternative, approximating silently, is exactly the failure mode essun exists to remove. section 4 shows a recorded case: ASTRA-sim exposes no byte accounting, so we record O-CONSERVATION as unsupported for that run instead of faking it.

Immutable runs, and classify-don't-tune. Every run stores its seed, topology, configuration hashes, and backend pins, and lands in an append-only evidence store. Determinism is structural: partitioned RNG streams, fixed tie-breaking, and integer byte counts. Both reference runs replay byte-for-byte from a fresh clone. When two fidelities disagree, we apply a discipline we call *classify-don't-tune*: minimize the counterexample, decompose the gap term by term, measure each coefficient with single-knob perturbations, and classify every difference as a modeling assumption, a unit or convention

Capability	Evidence artifact	Label
Test suite, 712 tests passing	`pytest`; `make ci-local`	IMPLEMENTED_UNVERIFIED
Deterministic replay: both reference runs byte-for-byte from a fresh clone; `make gate-demo` exits 0 on a cold tree	`experiments/vertical_slice/w1_decode_smoke.GOLDEN.md`; `Makefile`	VERIFIED
UEC 1.1 profile: 80 source anchors, 445 anchored requirements, 67 free parameters; owner expert review (task T011)	`experiments/uec/anchor_registry.yaml`;	VERIFIED
UnifiedBus paper-derived profile: 53 anchors from 5 public papers	`experiments/unifiedbus/`	IMPLEMENTED_UNVERIFIED
Cross-standard comparison (UEC vs. UnifiedBus, L0): delta 0 under shared parameters	sweep `9d07ec4a`; decision D-031	IMPLEMENTED_UNVERIFIED
L0 analytical family: ring, crossbar, 2D torus, recursive doubling, phase collectives, L0.5	`src/essun/backends/analytical/`	IMPLEMENTED_UNVERIFIED
ASTRA-sim analytical frontend, pinned `4ce9ecf`, Docker image with digest	`backends/pins/`; `backends/docker/`	IMPLEMENTED_UNVERIFIED
uet-htsim CMS backend (`uet_htsim_cms`), pinned `65ea315b`: four adapter stages, byte-identical replays; ratified third backend	`src/essun/backends/uet_htsim/`; ADR-0004	IMPLEMENTED_UNVERIFIED
csg-htsim packet frontend, pinned `67cbbbb1`: built + smoke-tested; seed-patch reproducible; no adapter yet	`backends/docker/astra-csg-runbook.md`	IMPLEMENTED_UNVERIFIED
Design-space atlas: 30/30 cells (5 rank classes $\times$ 6 algorithm families), E0 arithmetic facts	`docs/atlas/`	PROPOSED (E0)
Calibration: 23 published-measurement anchors staged; 26 model-vs-anchor records approved as E1 candidates	`docs/calibration/`	PROPOSED
Workloads: W1 dense TP decode, W2 prefill DAG (executed); W3 MoE, W5 PD/KV (gated)	`src/essun/workload/`	IMPLEMENTED_UNVERIFIED
Provenance-linked figure (L0 vs. ASTRA O-CCT)	`paper/figures/`	registered

Table 1: What we implement and test today. No capability carries the CALIBRATED label: we have fitted nothing against hardware. The 26 calibration candidates compare model identities against published anchor numbers at PROPOSED level; they change no shipped parameter value. Calibration execution is roadmap (section 6).

issue, or a capability gap. Parameter values never move during classification. Agreement bought by tuning is cosmetic; a classified gap is a map of what each simulator actually models.

3 What Exists Today

Every capability in essun carries an evidence label, and the labels form a strict ladder. `PROPOSED` means design only. `IMPLEMENTED_UNVERIFIED` means code exists and tests exercise it, with no check beyond the test suite. `VERIFIED` means a human or CI reproduced the result outside the author's working tree. `CALIBRATED` means we fitted the capability against hardware measurements; no capability carries this label yet. table 1 lists the current state of the repository.

The design-space atlas needs its scope stated plainly. Its 30 pages state arithmetic facts of the closed-form L0/L0.5 identities (traffic and completion identities, their exact-fraction deltas, and crossover payload sizes) under explicit free parameters, across rank classes 4–64 and six algorithm families; we measured nothing there on hardware, and none of it derives from a standard. The calibration candidates need the same plain statement. They are 26 PROPOSED records that compare model outputs against numbers published in measurement papers, and their headline findings are structural. The L0 torus identity is twist-blind by construction (model ratio exactly 1 where measurements read 1.63/1.31), the exact minimax fixed-bandwidth misfit across one anchor family is $\pm 7.02\%$, and three fabric-class derating brackets (0.712 xGMI, 0.80 NVLink-3, 0.93 B300) emerge from the published anchors themselves. These comparisons stand against published anchor numbers; they move no label and no shipped value.

We license the code under MIT, and the documentation, figures, and the project site under CC-BY-4.0 (`LICENSES.md`). The pinned external backends carry MIT or BSD-2-Clause licenses, recorded in `backends/pins/`. We keep the public repository lean by design (ADR-0003): historical working documents live only in git history, retrievable on demand, and we never rewrite history.

4 The Demo Story

From a clean clone, one command chains the whole story. The sequence below bootstraps the repository and then runs the gate end to end:

```
git clone https://github.com/li-ch/essun && cd essun
make bootstrap
make gate-demo
```

`make gate-demo` runs, in order: schema checks, the full test suite, both reference runs (the L0 golden and its pinned ASTRA-sim twin, replayed and compared byte-for-byte), a design-space sweep, figure registration, and the paper claims check over the paper directory and the README's gate section. Any failing leg stops the chain and prints that step's output. On a host without Docker, we skip the two Docker-dependent legs (the ASTRA-sim run and the figure render) and print the requirement instead, and the chain still exits 0. On a cold tree, the chain exits 0. The recorded runs carry identifiers, so every artifact the chain produces traces to the run that produced it.

When two simulators disagreed, we explained the whole gap. The centerpiece artifact is a cross-fidelity difference report. On the identical W1 smoke workload (a dependency-serialized chain of 6 ring AllReduce operations, $r = 8$ ranks, message size $M = 16384$ bytes per operation, free parameters $B = 64$ B/ns and $L = 200$ ns), the L0 ring identity and the pinned ASTRA-sim analytical frontend disagree on collective completion time by a factor of 7.404: per operation, 3248 ns against 24048 ns.

We applied classify-don't-tune to that gap. The decomposition splits each side's per-op completion time into transport terms, and we measured every coefficient by single-knob perturbation of the committed input set inside the pinned container, 22 probes in total. The result:

- The ring-latency term differs by $+19600$ ns/op, 94.23% of the gap. The recorded ASTRA-sim implementation profile serializes the ring into $s = 8$ dataset sub-streams, and each sub-stream pays the full $2(r-1)L$ latency. The L0 identity charges one pipelined ring pass, $2(r-1)L$ once.
- The endpoint term differs by $+1200$ ns/op, 5.77% of the gap. Every message in ASTRA-sim crosses a memory bus at a fixed 10 ns endpoint delay; L0 has no endpoint model by design.
- The serialized-byte term agrees bit-exactly at 448 ns on both sides. The disagreement lives exactly where the declared models differ, and agreement holds where the models should agree.

The per-op gap is 20800 ns, and the decomposition accounts for all of it: the residual is 0 ns. We changed no parameter to obtain any of these numbers, and both immutable run records stand untouched. We recorded the capability gap alongside the numbers: ASTRA-sim's output exposes no byte accounting, so we record the O-CONSERVATION observable as unsupported for that run.

We consider this decomposition the clearest statement of what essun is for. Any two simulators can disagree, and the raw disagreement is cheap. Producing a classified, measured, reproducible account of the disagreement, with every nanosecond attributed to a declared modeling difference, is the step we have not seen in the one-off comparisons we examined, and it is exactly the step we automated in essun.

5 Honest Limits

We have calibrated nothing in essun against hardware. The CALIBRATED evidence label exists, and no capability carries it. The 26 approved calibration candidates hold PROPOSED status: they compare model identities against numbers published in measurement papers, and their headline findings are structural statements about where the models and the published anchors disagree, namely the torus twist-blindness falsifier, the $\pm 7.02\%$ minimax misfit, and the three fabric-class derating brackets (0.712/0.80/0.93). No shipped value changed to obtain any of them. Executing fits against hardware measurements is roadmap work. Until that runs, essun produces internally consistent, traceable numbers, and the distance between those numbers and any physical fabric is unknown.

UALink and CXL are absent, and the absence is deliberate: their specification documents are license-restricted. essun ingests a standard only after license review, and neither standard cleared that gate. The UnifiedBus profile in the repository is paper-derived: we built it as a design profile from 5 public papers with 53 anchors, and we label it as such everywhere it appears. It is not a normative UnifiedBus specification profile.

The executable workload coverage spans two families. W1 is dense tensor-parallel decode, and W2 is prefill with a dependency DAG. Generators for two further regimes also exist, W3 mixture-of-experts dispatch/combine and W5

prefill/decode KV transfer, but no registered backend executes their operation types: the capability gates reject such manifests loudly, by design. Other regimes (serving feedback, fault events) are future work.

We use the phrase *implemented and tested* precisely. It means: code exists, tests exercise it, and the 712-test suite passes in the local CI chain (`make ci-local`). It does not mean agreement with hardware, and it does not mean agreement with other simulators beyond the recorded comparisons in the evidence store. Where a reader might assume verification, the evidence labels in table 1 state the actual level.

Counterexample-guided refinement (CGFR) influenced the design: the difference taxonomy we used to classify the 7.404x gap comes from the CGFR validation plan. The automated refinement loop itself is design on paper, with no implementation yet. We have not run blind evaluation of simulator outputs against ground truth.

6 Roadmap

The next steps are concrete, and none of them is done yet. Calibration execution will turn the 26 approved E1 candidates into a parameter-range-narrowing proposal: run the L0 family and the frontend backends against the staged anchors, report per-anchor error with full run records, and either move specific capabilities from IMPLEMENTED_UNVERIFIED toward CALIBRATED or document why they do not move. That adjudication will be its own decision, and the classify-don't-tune discipline binds it: the twist-blindness and derating-bracket findings serve as inputs that narrow parameter ranges, and no finding becomes a tuning knob.

The second normative standard will enter the profile library through the same gate every standard passes: license review, then page-hashed anchors, then requirement objects, then reconciliation against backend assumptions. UALink and CXL are the candidates; whether they clear licensing is an open question, and the license gate decides.

Backend coverage widens next. We will wire the csg-htsim packet frontend behind an adapter so it can produce run records, and we will run the scheduled cross-backend attribution probe that cross-checks the ratified uet-htsim path against the other engines. Executing the gated W3 mixture-of-experts and W5 prefill/decode KV generators needs L1/L2 backends, and the serving-feedback slice follows. The CGFR loop will move from design to implementation, using classified counterexamples like the 7.404x decomposition as its input.

We license the repository's code under MIT and its content under CC-BY-4.0, at `https://github.com/li-ch/essun`. The demo chain in section 4 offers the fastest check we know for any claim in this paper against the artifacts: clone, bootstrap, run the gate, and read the evidence store it produces.

References

[1] ASTRA-sim Project. ASTRA-sim 2.0. `https://github.com/astra-sim/astra-sim`, 2026. Pinned at commit 4ce9ecf (MIT license); analytical frontend integrated via Docker.

[2] NVIDIA. NVLink and NVSwitch: The building blocks of advanced multi-GPU communication. Whitepaper, 2023.

[3] Ultra Ethernet Consortium. Ultra Ethernet Specification 1.1. `https://ultraethernet.org/wp-content/uploads/sites/20/2026/10/UE-Specification-1.1.pdf`, 2026. 764 pages; profile built from this document with 80 page-hashed source anchors.

[4] Ultra Ethernet Consortium Transport WG. uet-htsim. `https://github.com/ultraethernet/uet-htsim`, 2026. Pinned at commit 65ea315b (BSD-2-Clause); packet-level path made reproducible with a recorded seed patch.